Lecture 13: Sorting Algorithms & Binary Search

Is lecture mein hum teen fundamental algorithms cover karenge: `Bubble Sort` , `Insertion Sort` , aur `Binary Search` . Ye algorithms `LeetCode 912` aur `LeetCode 704` ke solutions hain. Inki internal working, complexity analysis, aur interview-relevant insights ko detail mein samjhenge.

✦ ✦ ✦

Bubble Sort

> PATTERN
>
> **Sorting** — Bubble Sort, Comparison-Based, In-Place, Stable

Introduction

Bubble Sort ek simple `comparison-based sorting algorithm` hai. Ismein adjacent elements ko compare kiya jata hai aur agar wo wrong order mein hain toh swap kar diya jata hai. Har pass ke baad, sabse bada element apni correct position pe pahunch jata hai array ke end mein. Ye process tab tak repeat hoti hai jab tak poora array sorted na ho jaye.

> *Key Insight: Bubble Sort ka har pass `largest unsorted element` ko end mein "bubble up" kar deta hai, jaise paani mein hawa ka bubble upar ki taraf jaata hai.*

The Analogy

Socho aapke paas ek line mein students khade hain unki height ke hisaab se. Agar bagal wala student zyada lamba hai, toh wo aage chala jata hai. Har round ke baad sabse lamba student apni correct position pe pahunch jata hai end mein. Isi tarah Bubble Sort mein har pass ke baad largest element apni final position pe settle ho jata hai.

How It Works Internally

1. Algorithm total `n-1 passes` chalata hai.
2. Har pass mein, `adjacent elements` ko compare kiya jata hai.

3. Agar left element right element se bada hai, toh dono ko `swap` kar diya jata hai.
4. Har pass ke end mein, largest unsorted element apni correct sorted position pe pahunch jata hai.
5. Ye process tab tak repeat hoti hai jab tak poora array sorted na ho jaye.

> **WORKFLOW**
>
> **Pass 1:** Largest element index n-1 pe pahunchta hai.
> **Pass 2:** Second largest element index n-2 pe pahunchta hai.
> **Pass i:** i-th largest element apni correct position pe pahunchta hai.
> **Total:** n-1 passes ke baad array completely sorted hota hai.

Algorithm Steps

1. Outer loop `i = 0` se `n - 2` tak chalao.
2. Inner loop `j = 0` se `n - i - 2` tak chalao.
3. Compare `nums[j]` aur `nums[j + 1]`.
4. Agar `nums[j] > nums[j + 1]`, toh swap karo.
5. Return sorted `nums`.

Code Implementation

```cpp
// LeetCode 912 - Bubble Sort Solution
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Total n-1 passes
        for (int i = 0; i < n - 1; i++) {

            // Compare adjacent elements
            for (int j = 0; j < n - i - 1; j++) {

                // Swap if elements are in wrong order
                if (nums[j] > nums[j + 1]) {
                    swap(nums[j], nums[j + 1]);
                }
            }
        }

        return nums;
    }
};
```

Complexity Analysis

Case	Time Complexity	Reason
Best	$O(n^2)$	Without optimization, saare passes execute hote hain even if array already sorted hai.
Average	$O(n^2)$	Multiple comparisons aur swaps required hain har element ke liye.
Worst	$O(n^2)$	Reverse sorted array mein maximum swaps required hote hain.
Space	$O(1)$	In-place sorting hai, koi extra array use nahi hota.

PERFORMANCE NOTE

Optimization: Agar kisi pass mein `koi swap nahi hota`, iska matlab array already sorted hai. Ek `swapped flag` use karke early exit possible hai, jisse **Best Case O(n)** ban jata hai.

Common Mistakes & Edge Cases

EDGE CASE

- **Inner loop boundary:** `j < n - i - 1` hona chahiye, `j < n - 1` nahi. Last i elements already sorted hain, unhe dubara compare karne ki zaroorat nahi.
- **Empty array / Single element:** Directly return karo, loops execute nahi honge.
- **Duplicate elements:** Bubble Sort `stable` hai, equal elements ka relative order maintain hota hai.

IMPORTANT

`Bubble Sort practical applications mein rarely use hota hai.` Iska main purpose sorting fundamentals samajhna hai. Real-world mein `Quick Sort`, `Merge Sort`, ya `Tim Sort` use hote hain.

INTERVIEW TIP

Bubble Sort ko sirf **educational purpose** ke liye yaad rakho. Interview mein directly nahi mangta, lekin iski working principle clear honi chahiye. `Optimization wala variation` (swapped flag) zaroor batana — ye dikhata hai ki aap edge cases sochte hain.

KEY TAKEAWAYS

- Bubble Sort `stable` sorting algorithm hai.
- In-place sorting hai, space **O(1)**.
- Naive implementation mein time complexity **O(n²)** hai.
- Optimization se best case **O(n)** ban sakta hai.
- Har pass mein largest element end mein settle hota hai.

✦ ✦ ✦

Insertion Sort

PATTERN

Sorting — Insertion Sort, Comparison-Based, In-Place, Stable, Adaptive

Introduction

Insertion Sort mein array ko do parts mein divide kiya jata hai: `sorted part` aur `unsorted part`. Pehla element already sorted maana jata hai. Har iteration mein next element ko uthaya jata hai aur sorted part mein uski correct position pe insert kiya jata hai. Ye process array ke end tak repeat hoti hai.

> ***Key Insight:*** *Insertion Sort* `left side ko hamesha sorted maintain karta hai` *aur right side se elements uthata rehta hai. Har step mein sorted portion grow hota hai.*

The Analogy

Jaise card game mein player ek ek card uthata hai aur apne haath mein already sorted cards ke beech mein sahi jagah pe rakh deta hai. Har baar naya card aata hai, aur player usse compare karta hai already sorted cards se aur sahi position pe slide kar deta hai. Waise hi Insertion Sort kaam karta hai.

How It Works Internally

1. Index `0` ko already sorted maano.
2. Index `1` se last tak iterate karo.
3. Current element ko previous elements se compare karo.
4. Agar current element chota hai, toh swap karte jao left ki taraf.
5. Correct position milne par `break` karo.
6. Ye process har element ke liye repeat karo.

WORKFLOW

Iteration i: Element at index i ko sorted part [0, i-1] mein insert karo.
Comparison: nums[j] ko nums[j-1] se compare karo.
Shift: Agar nums[j] < nums[j-1], swap karo aur j-- karo.
Stop: Jab nums[j] >= nums[j-1] ya j == 0, break karo.

Algorithm Steps

1. `i = 1` se `n - 1` tak loop chalao.
2. `j = i` se 1 tak decrement karo.
3. Agar `nums[j] < nums[j - 1]`, toh swap karo.
4. Nahi toh break — correct position mil gayi.
5. Return sorted array.

Code Implementation

```cpp
// LeetCode 912 - Insertion Sort Solution
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Assume first element is already sorted
        for (int i = 1; i < n; i++) {

            // Insert current element into sorted part
            for (int j = i; j > 0; j--) {

                // Move current element towards left
                if (nums[j] < nums[j - 1]) {
                    swap(nums[j], nums[j - 1]);
                }

                // Correct position found
                else {
                    break;
                }
            }
        }

        return nums;
    }
};
```

Complexity Analysis

Case	Time Complexity	Reason
Best	$O(n)$	Array already sorted hai, sirf ek comparison per element required hai.
Average	$O(n^2)$	Elements ko partially shift karna padta hai.
Worst	$O(n^2)$	Reverse sorted array mein har element beginning tak shift hota hai.
Space	$O(1)$	In-place sorting hai, koi extra array use nahi hota.

BEST PRACTICE

Standard Insertion Sort implementation `shifting` use karta hai instead of swapping: current element ko store karo, larger elements ko right shift karo, aur stored element ko correct gap mein daalo. Isse number of assignments kam hote hain. Lekin swap-based approach bhi **logically correct** hai aur interviews mein accept hota hai.

Common Mistakes & Edge Cases

EDGE CASE

- **j > 0 condition:** Inner loop mein hamesha `j > 0` check karo taaki index out of bounds na ho.
- **Nearly sorted arrays:** Insertion Sort nearly sorted data pe bahut efficient hai. Ye fact miss mat karna.
- **Duplicates:** Insertion Sort stable hai, equal elements ka order maintain hota hai.

INTERVIEW TIP

Insertion Sort **small datasets** aur **nearly sorted arrays** ke liye best choice hai. Ye `adaptive` sorting algorithm hai — already sorted data pe bahut fast kaam karta hai. Real-world mein ye `TimSort` aur `IntroSort` mein hybrid component ke roop mein use hota hai.

KEY TAKEAWAYS

- Insertion Sort `adaptive` aur `stable` hai.
- Best case **O(n)**, worst case **O(n²)**.
- Left side hamesha sorted maintain hoti hai.
- Real-world mein small arrays ke liye use hota hai (Timsort, Introsort).
- Nearly sorted data ke liye highly efficient hai.

Binary Search

PATTERN

Searching — Binary Search, Divide & Conquer, Sorted Array Required

Introduction

Binary Search ek searching algorithm hai jo `sirf sorted arrays` pe kaam karta hai. Har comparison ke baad search space ko `half` kar deta hai. Isliye ye Linear Search se exponentially fast hai. Is algorithm ko samajhna DSA ke liye fundamental hai kyunki iske variations bahut se advanced problems mein use hote hain.

> *Key Insight: Binary Search har step mein search space ko `half kar deta hai` . Isliye 1 million elements mein sirf **~20 comparisons** mein target mil sakta hai, jabki Linear Search mein 1 million comparisons lag sakte hain.*

The Analogy

Socho aapko dictionary mein ek word dhoondhna hai. Aap beech se khulte hain. Agar word alphabetically aage hai, toh pehli aadhi dictionary discard kar dete hain. Agar peeche hai, toh aadhi dictionary discard kar dete hain. Baar baar aadha karke jaldi pahunch jaate hain. Binary Search exactly aise hi kaam karta hai.

How It Works Internally

1. Do pointers initialize karo: `low = 0` , `high = n - 1` .
2. Tab tak chalao jab tak `low <= high`:
3. Mid calculate karo: `mid = low + (high - low) / 2` (overflow se bachne ke liye).
4. Three cases:
    - **Found:** `nums[mid] == target` → return `mid`.
    - **Go Right:** `nums[mid] < target` → `low = mid + 1`.
    - **Go Left:** `nums[mid] > target` → `high = mid - 1`.
5. Loop end hone par target nahi mila, return `-1` .

WORKFLOW

Step 1: low = 0, high = n-1 initialize karo.
Step 2: mid = low + (high - low) / 2.
Step 3: nums[mid] ko target se compare karo.
Step 4: Equal? Return mid. Chota? Right jao. Bada? Left jao.
Step 5: low > high hone par return -1.

Visual Diagram — Search Space Elimination

Array: [-1, 0, 3, 5, 9, 12] | Target: 9

Step 1: mid=2, value=3. 3 < 9, so left half discarded. Search [5, 9, 12]

Step 2: mid=4, value=9. 9 == 9, Found at index 4!

Algorithm Steps

1. `low = 0, high = n - 1` initialize karo.
2. `while (low <= high)` loop chalao.
3. Mid calculate karo: `low + (high - low) / 2` (overflow safe).
4. Three cases handle karo: found, go right, go left.
5. Target nahi mila toh `return -1`.

Code Implementation

```cpp
// LeetCode 704 - Binary Search Solution
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {

            // Prevents integer overflow
            int mid = low + (high - low) / 2;

            // Target Found
            if (nums[mid] == target) {
                return mid;
            }

            // Search Right Half
            else if (nums[mid] < target) {
                low = mid + 1;
            }

            // Search Left Half
            else {
                high = mid - 1;
            }
        }

        // Target doesn't exist
        return -1;
    }
};
```

Complexity Analysis

Case	Time Complexity	Reason
Best	O(1)	Target first mid pe hi mil jaye.
Average	O(log n)	Har step mein search space half hota hai.
Worst	O(log n)	Target last comparison pe mile ya array mein na ho.
Space	O(1)	Sirf low, high, mid variables use hote hain.

Common Mistakes & Edge Cases

EDGE CASE

- **Integer Overflow:** `(low + high) / 2` use mat karo. Bade arrays mein low + high overflow ho sakta hai. Hamesha `low + (high - low) / 2` use karo.
- **While condition:** `low <= high` use karo, `low < high` nahi. Single element bhi check hona chahiye.
- **Boundary updates:** `low = mid + 1` aur `high = mid - 1` karo. `low = mid` se infinite loop ho sakta hai.
- **Empty array:** Directly return -1.

IMPORTANT

Binary Search sirf sorted arrays pe kaam karta hai . Unsorted data pe apply karne se galat result milega. Hamesha pehle sorting condition verify karo.

INTERVIEW TIP

Binary Search ka template **rote learn mat karo**, logic samjho. Three cases hain: equal, go left, go right. Overflow-safe mid calculation hamesha mention karna. Binary Search sirf arrays pe nahi, **rotated arrays**, **sorted matrices**, aur **answer space search** (Binary Search on Answer) mein bhi use hota hai.

KEY TAKEAWAYS

- Binary Search sirf sorted data pe kaam karta hai.
- Time complexity **O(log n)** hai.
- Overflow-safe mid calculation zaroori hai: `low + (high - low) / 2`.
- Template-based approach se variations easily solve hote hain.
- Real-world mein databases, file systems, aur competitive programming mein extensively use hota hai.

✦ ✦ ✦

Comparative Summary

Algorithm	Best Time	Average Time	Worst Time	Space	Stable
Bubble Sort	$O(n^2)$ / $O(n)$*	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	N/A

* Bubble Sort with swapped flag optimization

LECTURE TAKEAWAYS

- **Bubble Sort** sorting fundamentals samajhne ke liye best hai, lekin practical use mein rarely aata hai.
- **Insertion Sort** small arrays aur nearly sorted data ke liye highly efficient hai.
- **Binary Search** har developer ko rote learn hona chahiye — iske bina DSA incomplete hai.
- In algorithms ko **dry run** karna mat bhoolna. Paper pe chalao, tabhi true understanding aayegi.
